

แบบฝึกหัดการออกแบบอัลกอริทึมแบบเวียนเกิดและการวิเคราะห์ Big-O

คำชี้แจง

ให้นักศึกษาเขียนโปรแกรมภาษา Java เพื่อแก้ปัญหาแต่ละข้อ โดยแต่ละข้อจะต้องมีอัลกอริทึมอย่างน้อย 2 วิธี และวิเคราะห์ประสิทธิภาพของแต่ละอัลกอริทึม

สิ่งที่ต้องส่งในแต่ละข้อประกอบด้วย

1. คำอธิบายแนวคิดของอัลกอริทึมแต่ละวิธี
2. Pseudocode หรือผังขั้นตอนการทำงาน
3. โปรแกรมภาษา Java ที่สามารถทำงานได้จริง
4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์
5. การวิเคราะห์ Time Complexity
6. การวิเคราะห์ Space Complexity
7. การเปรียบเทียบข้อดีและข้อจำกัดของแต่ละอัลกอริทึม
8. สรุปว่าอัลกอริทึมใดเหมาะสมกว่าภายใต้เงื่อนไขใด

ข้อ 1 การกลับลำดับสตริง

กำหนดสตริง s ให้นักศึกษาเขียนโปรแกรมเพื่อสร้างสตริงใหม่ที่มีลำดับตัวอักษรย้อนกลับจากสตริงเดิม
ตัวอย่าง

Input: pots&pans

Output: snap&stop

ให้ออกแบบอย่างน้อย 2 อัลกอริทึม ได้แก่

อัลกอริทึมที่ 1: Recursive Algorithm

เขียนเมธอดแบบเวียนเกิด โดยนำตัวอักษรตัวสุดท้ายมาต่อกับผลลัพธ์จากการเรียกเมธอดกับสตริงส่วนที่เหลือ

ชื่อเมธอด **static String reverseRecursive(String s)**

1. คำอธิบายแนวคิด

อัลกอริทึมนี้ใช้แนวคิดการเวียนเกิด (Recursion) โดยนำตัวอักษรตัวสุดท้ายของสตริงมาต่อกับผลลัพธ์ที่ได้จากการเรียกเมธอดซ้ำกับสตริงส่วนที่เหลือ

ตัวอย่าง

pots

= s + reverseRecursive(pot)

= s + t + reverseRecursive(po)

= s + t + o + reverseRecursive(p)

= stop

2. Pseudocode

```
reverseRecursive(s)

if length(s) <= 1
    return s

return last character of s
+ reverseRecursive(s without last character)
```

3. โปรแกรมภาษา Java

```
import java.util.Scanner;

public class ReverseStringRecursive {

    static String reverseRecursive(String s){
        if(s.length() <= 1){
            return s;
        }

        return s.charAt(s.length()-1)
            + reverseRecursive(s.substring(0, s.length()-1));
    }

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        System.out.print("Input:");
        String s = sc.nextLine();

        System.out.println("Output:" + reverseRecursive(s));

        sc.close();
    }
}
```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input : pots&pans

Output : snap&stop

5. การวิเคราะห์ Time Complexity

ในแต่ละการเรียกเมธอดมีการสร้าง String ใหม่จากการต่อข้อความด้วยเครื่องหมาย + ดังนั้น

$$T(n) = T(n-1) + O(n)$$

สรุป

$$\text{Time Complexity} = O(n^2)$$

6. การวิเคราะห์ Space Complexity

มีการเรียกเมธอดซ้อนกันตามจำนวนตัวอักษร

$$\text{Space Complexity} = O(n)$$

7. ข้อดีและข้อจำกัด

ข้อดี

- โค้ดสั้นและเข้าใจง่าย
- แสดงแนวคิด Recursion ได้ชัดเจน
- เหมาะสำหรับการเรียนรู้การแก้ปัญหาแบบเวียนเกิด

ข้อจำกัด

- ใช้หน่วยความจำสำหรับ Call Stack
- ประสิทธิภาพลดลงเมื่อสตริงมีขนาดใหญ่
- อาจเกิด Stack Overflow ได้

8. สรุป

เหมาะกับการศึกษาแนวคิด Recursion และการทำงานกับข้อมูลขนาดเล็กถึงปานกลาง

อัลกอริทึมที่ 2: Iterative Algorithm

ใช้ลูปเพื่ออ่านข้อความจากตำแหน่งสุดท้ายย้อนกลับไปยังตำแหน่งแรก

ชื่อเมธอด **static String reverseliterative(String s)**

1. คำอธิบายแนวคิด

ใช้อัลกอริทึมแบบวนรอบ (Iteration) โดยอ่านตัวอักษรจากตำแหน่งสุดท้ายย้อนกลับมาจนถึงตำแหน่งแรก และนำมาต่อกันเป็นสตริงใหม่

2. Pseudocode

```
reverseIterative(s)
result = ""
for i = length(s)-1 down to 0
    result = result + s[i]
return result
```

3. โปรแกรมภาษา Java

```
import java.util.Scanner;

public class ReverseStringIterative {

    static String reverseIterative(String s){

        StringBuilder result = new StringBuilder();

        for(int i = s.length()-1; i >=0; i--){
            result.append(s.charAt(i));
        }

        return result.toString();
    }

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        System.out.print("Input:");
        String s = sc.nextLine();

        System.out.println("Output:" + reverseIterative(s));

        sc.close();
    }
}
```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input : pots&pans

Output : snap&stop

5. การวิเคราะห์ Time Complexity

ลูปทำงานผ่านตัวอักษรทุกตัวเพียง 1 ครั้ง

Time Complexity = $O(n)$

6. การวิเคราะห์ Space Complexity

ใช้พื้นที่เก็บผลลัพธ์ใน StringBuilder

Space Complexity = $O(n)$

7. ข้อดีและข้อจำกัด

ข้อดี

- ทำงานรวดเร็ว
- ไม่ใช้ Call Stack
- เหมาะกับข้อมูลขนาดใหญ่
- ไม่เสี่ยงต่อ Stack Overflow

ข้อจำกัด

- โค้ดยาวกว่า Recursive เล็กน้อย
- ต้องมีตัวแปรเก็บผลลัพธ์เพิ่มเติม

8. สรุป

เหมาะสำหรับการใช้งานจริง โดยเฉพาะเมื่อต้องจัดการสตริงที่มีขนาดใหญ่
งานวิเคราะห์

ให้นักศึกษาวิเคราะห์และเปรียบเทียบ

ให้นักศึกษาทดสอบกับสตริงขนาดประมาณ

10, 100, 1,000 และ 10,000 ตัวอักษร

- จำนวนครั้งที่แต่ละอัลกอริทึมประมวลผลตัวอักษร

ขนาดสตริง	Recursive	Iterative
10 ตัวอักษร	10 ครั้ง	10 ครั้ง
100 ตัวอักษร	100 ครั้ง	100 ครั้ง
1,000 ตัวอักษร	1,000 ครั้ง	1,000 ครั้ง
10,000 ตัวอักษร	10,000 ครั้ง	10,000 ครั้ง

ทั้งสองวิธีต้องอ่านตัวอักษรทุกตัวอย่างน้อย 1 ครั้ง

- Time Complexity

อัลกอริทึม	Time Complexity
Recursive	$O(n^2)$
Iterative (StringBuilder)	$O(n)$

- Space Complexity

อัลกอริทึม	Space Complexity
Recursive	$O(n)$

Iterative	$O(n)$
-----------	--------

- ผลกระทบจากการต่อสตริงด้วยเครื่องหมาย + ตัวอย่าง

```
String result = "";
result = result + "A";
result = result + "B";
result = result + "C";
```

เนื่องจาก String เป็น Immutable ทุกครั้งที่ใช้ + จะสร้างอ็อบเจกต์ใหม่ ทำให้เสียเวลาและใช้หน่วยความจำมากขึ้น

เมื่อจำนวนตัวอักษรเพิ่มขึ้นเป็น 1,000 หรือ 10,000 ตัวอักษร ประสิทธิภาพจะลดลงอย่างชัดเจน

- ความแตกต่างระหว่างการใช้ String และ StringBuilder

String

```
String text = "ABC";
text = text + "D";
```

- **Immutable**
- สร้าง **Object** ใหม่ทุกครั้งที่แก้ไขข้อมูล
- ช้ากว่าเมื่อมีการต่อข้อความหลายครั้ง

StringBuilder

```
StringBuilder sb = new StringBuilder();
sb.append("A");
sb.append("B");
sb.append("C");
```

- **Mutable**
- แก้ไขข้อมูลในอ็อบเจกต์เดิมได้
- ทำงานได้เร็วกว่า
- เหมาะกับงานที่มีการต่อข้อความจำนวนมาก

สรุปเปรียบเทียบอัลกอริทึม

หัวข้อ	Recursive	Iterative
วิธีการทำงาน	Recursion	Loop
Time Complexity	$O(n^2)$	$O(n)$
Space Complexity	$O(n)$	$O(n)$
ความเร็ว	ช้ากว่า	เร็วกว่า
Stack Overflow	อาจเกิดขึ้น	ไม่เกิด
เหมาะกับข้อมูลขนาดใหญ่	ไม่ค่อยเหมาะ	เหมาะ

สรุปสุดท้าย

สำหรับการใช้งานจริง Iterative Algorithm ร่วมกับ StringBuilder เหมาะสมที่สุด เพราะมี Time Complexity เท่ากับ $O(n)$ ทำงานได้รวดเร็วและรองรับข้อมูลขนาดใหญ่ได้ดีกว่า ส่วน Recursive Algorithm เหมาะกับการศึกษาแนวคิดการเวียนเกิดและการวิเคราะห์อัลกอริทึมเป็นหลัก

ข้อ 2 การตรวจสอบ Palindrome

กำหนดสตริง s ให้นักศึกษาเขียนโปรแกรมเพื่อตรวจสอบว่าสตริงดังกล่าวเป็น Palindrome หรือไม่ Palindrome คือสตริงที่อ่านจากซ้ายไปขวาและขวาไปซ้ายแล้วได้ข้อความเดียวกัน

ตัวอย่าง

racecar → true

level → true

algorithm → false

gohangasalamiimalasagnahog → true

ให้ออกแบบอย่างน้อย 2 อัลกอริทึม ได้แก่

อัลกอริทึมที่ 1: Reverse and Compare

สร้างสตริงย้อนกลับก่อน แล้วจึงเปรียบเทียบกับสตริงเดิม

ชื่อเมธอด

```
static boolean isPalindromeByReverse(String s)
```


1. คำอธิบายแนวคิด

วิธีนี้จะสร้างสตริงที่กลับลำดับจากสตริงเดิมก่อน จากนั้นนำมาเปรียบเทียบกับสตริงต้นฉบับก่อนตรวจสอบจะทำการแปลงสตริงให้อยู่ในรูปมาตรฐาน โดย

- แปลงเป็นตัวพิมพ์เล็กทั้งหมด
- ลบช่องว่าง
- ลบเครื่องหมายวรรคตอน

หากสตริงเดิมและสตริงที่กลับลำดับเหมือนกัน แสดงว่าเป็น Palindrome

2. Pseudocode

```
isPalindromeByReverse(s)

cleanString(s)

reverse = reverse(cleanString)

if reverse equals cleanString
return true
else
return false
```

3. โปรแกรมภาษา Java

```
import java.util.Scanner;

public class PalindromeReverse {

    static boolean isPalindromeByReverse(String s){

        String cleaned =
s.replaceAll("[^a-zA-Z0-9]", "")
.toLowerCase();

        StringBuilder reverse = new StringBuilder();

        for(int i = cleaned.length()-1; i >=0; i--){
            reverse.append(cleaned.charAt(i));
        }

        return cleaned.equals(reverse.toString());
    }

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        System.out.print("Input:");
        String s = sc.nextLine();

        System.out.println(isPalindromeByReverse(s));

        sc.close();
    }
}
```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input : racecar

Output : true

Input : algorithm

Output : false

Input : A man, a plan, a canal: Panama

Output : true

5. Time Complexity

- ทำความสะอาดสตริง $O(n)$
- กลับลำดับสตริง $O(n)$
- เปรียบเทียบสตริง $O(n)$

ดังนั้น

$$\text{Time Complexity} = O(n)$$

6. Space Complexity

ต้องสร้างสตริงใหม่และสตริงที่กลับด้าน

$$\text{Space Complexity} = O(n)$$

7. ข้อดีและข้อจำกัด

ข้อดี

- เข้าใจง่าย
- เขียนโปรแกรมได้ง่าย
- เหมาะกับผู้เริ่มต้น

ข้อจำกัด

- ต้องสร้างสตริงกลับด้านเพิ่ม
- ใช้หน่วยความจำมากกว่า Two-Pointer

8. สรุป

เหมาะสำหรับงานทั่วไปที่ต้องการโค้ดอ่านง่ายและดูแลรักษาง่าย

อัลกอริทึมที่ 2: Recursive Two-Pointer

เปรียบเทียบตัวอักษรตำแหน่งซ้ายสุดและขวาสุด หากเหมือนกัน ให้ตรวจสอบตัวอักษรคู่ถัดไปด้วยการเรียกเมธอดแบบเวียนเกิด

ชื่อเมธอด

static boolean isPalindromeRecursive(String s, int left, int right)

1. คำอธิบายแนวคิด

เปรียบเทียบตัวอักษรจากด้านซ้ายและด้านขวาพร้อมกัน

- ถ้าไม่เท่ากัน คืนค่า false ทันที
- ถ้าเท่ากัน ให้ตรวจสอบตัวอักษรคู่ถัดไป
- เมื่อตัวชี้ทั้งสองมาพบกันหรือไขว้กัน แสดงว่าเป็น Palindrome

วิธีนี้สามารถหยุดทำงานได้ทันทีเมื่อพบความแตกต่าง

2. Pseudocode

```
isPalindromeRecursive(s, left, right)

if left >= right
return true

if s[left] != s[right]
return false

return isPalindromeRecursive(
s,
left+1,
right-1)
```

3. โปรแกรมภาษา Java

```
import java.util.Scanner;

public class PalindromeRecursive {

    static boolean isPalindromeRecursive(String s,
    int left,
    int right){

        if(left >= right){
            return true;
        }

        if(s.charAt(left) != s.charAt(right)){
            return false;
        }

        return isPalindromeRecursive(
            s,
            left +1,
            right -1);
    }

    public static void main(String[] args){

        Scanner sc =new Scanner(System.in);

        System.out.print("Input:");
```

```
String input = sc.nextLine();

String cleaned =
input.replaceAll("[^a-zA-Z0-9]", "")
.toLowerCase();

boolean result =
isPalindromeRecursive(
cleaned,
0,
cleaned.length()-1);

System.out.println(result);

sc.close();
}
}
```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input : level

Output : true

Input : algorithm

Output : false

Input : A man, a plan, a canal: Panama

Output : true

5. Time Complexity

กรณีแย่ที่สุด

ต้องตรวจสอบตัวอักษรประมาณครึ่งหนึ่งของสตริง

Time Complexity = $O(n)$

6. Space Complexity

เกิด Recursive Call หลายชั้น

Space Complexity = $O(n)$

7. ข้อดีและข้อจำกัด

ข้อดี

- ไม่ต้องสร้างสตริงกลับด้าน
- หยุดการทำงานได้ทันทีเมื่อพบความแตกต่าง
- ใช้จำนวนการเปรียบเทียบน้อยลง

ข้อจำกัด

- มีค่าใช้จ่ายจาก **Recursive Call**
- อาจเกิด **Stack Overflow** หากสตริงยาวมาก

8. สรุป

เหมาะกับการลดจำนวนการตรวจสอบและสามารถหยุดทำงานได้เร็วเมื่อพบว่าข้อมูลไม่เป็น Palindrome

งานวิเคราะห์

ให้นักศึกษาวิเคราะห์และเปรียบเทียบ

- กรณีที่สตริงเป็น **Palindrome**

ตัวอย่าง

racecar

Reverse and Compare

- สร้างสตริงย้อนกลับทั้งหมด
- เปรียบเทียบครบทุกตัวอักษร

Recursive Two-Pointer

- เปรียบเทียบ r กับ r
- เปรียบเทียบ a กับ a
- เปรียบเทียบ c กับ c
- ถึงจุดกึ่งกลางแล้วจบ

ทั้งสองวิธีให้ผลลัพธ์

true

- กรณีที่ตัวอักษรคู่แรกไม่ตรงกัน

ตัวอย่าง

abcdef

Reverse and Compare

ยังต้องสร้างสตริงย้อนกลับทั้งหมดก่อน

fedcba

แล้วจึงเปรียบเทียบ

Recursive Two-Pointer

ตรวจเพียง

a กับ f

พบว่าไม่เท่ากัน **false** ทันที

- **Best-case Time Complexity**

อัลกอริทึม	Best Case
Reverse and Compare	$O(n)$
Recursive Two-Pointer	$O(1)$

เนื่องจาก Two-Pointer อาจพบความแตกต่างตั้งแต่คู่แรกแล้วหยุดทำงานทันที

- **Worst-case Time Complexity**

อัลกอริทึม	Worst Case
Reverse and Compare	$O(n)$
Recursive Two-Pointer	$O(n)$

- **Space Complexity**

อัลกอริทึม	Space Complexity
Reverse and Compare	$O(n)$
Recursive Two-Pointer	$O(n)$

- ความสามารถในการหยุดทำงานก่อนครบทุกตัวอักษร

เงื่อนไขเพิ่มเติม

ปรับโปรแกรมให้สามารถละเว้น

- ตัวพิมพ์เล็กและตัวพิมพ์ใหญ่
- ช่องว่าง
- เครื่องหมายวรรคตอน

ตัวอย่าง

A man, a plan, a canal: Panama

ควรให้ผลลัพธ์เป็น

true

อัลกอริทึม	หยุดก่อนครบทุกตัวได้หรือไม่
Reverse and Compare	ไม่ได้
Recursive Two-Pointer	ได้

ตัวอย่าง

abcdef

Reverse and Compare

ต้องสร้าง fedcba ก่อนเสมอ

Recursive Two-Pointer

$a \neq f$

หยุดทันที

สรุปเปรียบเทียบอัลกอริทึม

หัวข้อ	Reverse and Compare	Recursive Two-Pointer
แนวคิด	กลับสตริงแล้วเปรียบเทียบ	เปรียบเทียบหัวท้าย
Worst-case Time	$O(n)$	$O(n)$
Best-case Time	$O(n)$	$O(1)$
Space	$O(n)$	$O(n)$
หยุดก่อนครบทุกตัวได้	ไม่ได้	ได้
ความง่ายในการเขียน	ง่ายมาก	ปานกลาง

สรุปสุดท้าย

หากต้องการโค้ดที่เข้าใจง่าย ควรเลือก Reverse and Compare แต่หากต้องการประสิทธิภาพที่ดีกว่าในกรณีที่ข้อมูลไม่เป็น Palindrome ควรเลือก Recursive Two-Pointer เพราะสามารถหยุดการทำงานได้ทันทีเมื่อพบตัวอักษรที่ไม่ตรงกัน ทำให้ลดจำนวนการเปรียบเทียบได้มากในหลายกรณี

ข้อ 3 การเปรียบเทียบจำนวนสระและพยัญชนะ

กำหนดสตริงภาษาอังกฤษ s ให้นักศึกษาเขียนโปรแกรมเพื่อตรวจสอบว่าสตริงนั้นมีจำนวนสระมากกว่าจำนวนพยัญชนะหรือไม่

กำหนดให้สระภาษาอังกฤษ ได้แก่

a, e, i, o, u

ตัวอย่าง

Input: education

Vowels: 5

Consonants: 4

Result: true

ให้ออกแบบอย่างน้อย 2 อัลกอริทึม ได้แก่

อัลกอริทึมที่ 1: Recursive Counting

ตรวจสอบตัวอักษรทีละตัวด้วยการเรียกเมธอดแบบเวียนเกิด และส่งค่าจำนวนสระและพยัญชนะไปยังการเรียกครั้งถัดไป

ชื่อเมธอด

static boolean hasMoreVowelsRecursive(String s)

1. คำอธิบายแนวคิด

ใช้อัลกอริทึมแบบ Recursion ตรวจสอบตัวอักษรทีละตัว

- ถ้าเป็นสระ เพิ่มจำนวนสระ
- ถ้าเป็นพยัญชนะ เพิ่มจำนวนพยัญชนะ
- ส่งค่าที่นับได้ไปยังการเรียกเมธอดครั้งถัดไป
- หยุดเมื่ออ่านครบทุกตัวอักษร

2. Pseudocode

```
count(index, vowels, consonants)

if index == length
return

if current character is vowel
vowels++

elseif current character is consonant
consonants++

count(index + 1, vowels, consonants)
```

3. โปรแกรมภาษา Java

```
import java.util.Scanner;

public class VowelConsonantRecursive {

    static int vowels = 0;
    static int consonants = 0;

    static void countRecursive(String s, int index){

        if(index == s.length()){
            return;
        }

        char ch = Character.toLowerCase(s.charAt(index));

        if(Character.isLetter(ch)){

            if("aeiou".indexOf(ch) != -1){
                vowels++;
            } else {
                consonants++;
            }
        }

        countRecursive(s, index + 1);
    }

    static boolean hasMoreVowelsRecursive(String s){

        vowels = 0;
        consonants = 0;

        countRecursive(s, 0);

        System.out.println("Vowels:" + vowels);
        System.out.println("Consonants:" + consonants);

        return vowels > consonants;
    }

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        System.out.print("Input:");
        String s = sc.nextLine();
    }
}
```

```
System.out.println("Result:"  
+ hasMoreVowelsRecursive(s));  
  
sc.close();  
}  
}
```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input : education

Output : Vowels: 5

Consonants: 4

Result: true

5. การวิเคราะห์ Time Complexity

โปรแกรมตรวจสอบตัวอักษรทุกตัวเพียง 1 ครั้ง

Time Complexity = $O(n)$

6. การวิเคราะห์ Space Complexity

เกิด Recursive Call ตามจำนวนตัวอักษร

Space Complexity = $O(n)$

7. ข้อดีและข้อจำกัด

ข้อดี

- เหมาะสำหรับเรียนรู้ Recursion
- โค้ดแสดงขั้นตอนการนับได้ชัดเจน

ข้อจำกัด

- ใช้หน่วยความจำมากขึ้นจาก Call Stack
- อาจเกิด StackOverflowError เมื่อข้อมูลมีขนาดใหญ่

8. สรุป

เหมาะกับการศึกษาแนวคิด Recursion และการส่งค่าผ่านการเรียกเมธอดซ้ำ

อัลกอริทึมที่ 2: Iterative Counting

ใช้ลูปอ่านข้อความทุกตัว แล้วเพิ่มตัวนับสระหรือพยัญชนะ
ชื่อเมธอด

static boolean hasMoreVowelsIterative(String s)

เงื่อนไข

- ไม่นับตัวเลข
- ไม่นับช่องว่าง
- ไม่นับเครื่องหมายพิเศษ

- ไม่แยกตัวพิมพ์เล็กและตัวพิมพ์ใหญ่

1. คำอธิบายแนวคิด

ใช้ลูปอ่านตัวอักษรทุกตัวในสตริง

- ถ้าเป็นสระ เพิ่มตัวนับสระ
- ถ้าเป็นพยัญชนะ เพิ่มตัวนับพยัญชนะ
- เมื่อจบลูปให้นำจำนวนที่ได้มาเปรียบเทียบ

2. Pseudocode

```
vowels = 0
consonants = 0

for each character in string
    if vowel
        vowels++
    elseif consonant
        consonants++

return vowels > consonants
```

3. โปรแกรมภาษา Java

```
import java.util.Scanner;

public class VowelConsonantIterative {

    static boolean hasMoreVowelsIterative(String s){

        int vowels = 0;
        int consonants = 0;

        s = s.toLowerCase();

        for(int i = 0; i < s.length(); i++){

            char ch = s.charAt(i);

            if(Character.isLetter(ch)){

                if("aeiou".indexOf(ch) != -1){
                    vowels++;
                } else {
                    consonants++;
                }
            }
        }

        System.out.println("Vowels:" + vowels);
        System.out.println("Consonants:" + consonants);

        return vowels > consonants;
    }

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);
```

```
System.out.print("Input:");
String s = sc.nextLine();

System.out.println("Result:"
+ hasMoreVowelsIterative(s));

sc.close();
}
}
```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input : education

Output : Vowels: 5

Consonants: 4

Result: true

Input : computer

Output : Vowels: 3

Consonants: 5

Result: false

5. การวิเคราะห์ Time Complexity

วนลูปครบทุกตัวอักษรเพียงหนึ่งรอบ

Time Complexity = $O(n)$

6. การวิเคราะห์ Space Complexity

ใช้ตัวแปรนับเพียงไม่กี่ตัว

Space Complexity = $O(1)$

7. ข้อดีและข้อจำกัด

ข้อดี

- ทำงานรวดเร็ว
- ใช้หน่วยความจำต่ำ
- ไม่เกิด **StackOverflowError**

ข้อจำกัด

- ไม่ได้แสดงแนวคิด **Recursion**

8. สรุป

เหมาะสำหรับการใช้งานจริงและข้อมูลขนาดใหญ่

งานวิเคราะห์

ให้นักศึกษาวิเคราะห์

- **Time Complexity** ของทั้งสองวิธี

วิธี	Time Complexity
Recursive Counting	$O(n)$
Iterative Counting	$O(n)$

เนื่องจากทั้งสองอัลกอริทึมต้องตรวจสอบตัวอักษรทุกตัวอย่างน้อยหนึ่งครั้ง

- **Space Complexity** ของทั้งสองวิธี

วิธี	Space Complexity
Recursive Counting	$O(n)$
Iterative Counting	$O(1)$

Recursive ต้องใช้ Call Stack เพิ่มตามจำนวนตัวอักษร ส่วน Iterative ใช้ตัวแปรนับเพียงไม่กี่ตัว

- **จำนวน recursive calls**

หากสตริงมีความยาว n ตัว

$$\text{จำนวน Recursive Calls} = n + 1$$

ตัวอย่าง

จำนวนตัวอักษร	Recursive Calls
10	11
100	101
1,000	1,001
10,000	10,001

- ความเสี่ยงของ **StackOverflowError**

Recursive Counting

มีความเสี่ยงเนื่องจากทุกตัวอักษรจะสร้าง Call Stack ใหม่

ตัวอย่าง

ข้อความยาว **10,000** ตัวอักษร

อาจทำให้ Stack เต็มได้ในบางระบบ

Iterative Counting

ไม่มีความเสี่ยงเพราะใช้รูปแบบการเรียกเมธอดซ้ำ

- ขนาดข้อมูลที่เหมาะสมสำหรับแต่ละวิธี

Recursive Counting

เหมาะกับ ข้อมูลขนาดเล็กถึงปานกลาง 10 - 1,000 ตัวอักษร

Iterative Counting

เหมาะกับ ข้อมูลทุกขนาด 10 - 100,000+ ตัวอักษร

สรุปเปรียบเทียบอัลกอริทึม

หัวข้อ	Recursive Counting	Iterative Counting
วิธีการ	Recursion	Loop
Time Complexity	$O(n)$	$O(n)$
Space Complexity	$O(n)$	$O(1)$
จำนวน Calls	$n+1$	ไม่มี
ความเสี่ยง StackOverflowError	มี	ไม่มี
เหมาะกับข้อมูลขนาดใหญ่	ไม่ค่อยเหมาะ	เหมาะ
ความง่ายในการทำความเข้าใจ	ปานกลาง	ง่าย

สรุปสุดท้าย

แม้ว่าทั้ง **Recursive Counting** และ **Iterative Counting** จะมี Time Complexity เท่ากันคือ

$O(n)$ แต่ **Iterative Counting** มี Space Complexity ดีกว่า (**$O(1)$**) และไม่มีความเสี่ยงต่อ

StackOverflowError จึงเหมาะสำหรับการใช้งานจริงและข้อมูลขนาดใหญ่ ส่วน **Recursive Counting**

เหมาะสำหรับการเรียนรู้แนวคิดการเวียนเกิดและการวิเคราะห์การเรียกเมธอดแบบ Recursion มากกว่า

ข้อ 4 การจัดกลุ่มจำนวนคู่และจำนวนคี่

กำหนดอาร์เรย์จำนวนเต็ม A ให้นักศึกษาเขียนโปรแกรมจัดตำแหน่งสมาชิกใหม่ โดยให้จำนวนคู่ทั้งหมดอยู่ก่อนจำนวนคี่

ตัวอย่าง

Input:

[7, 2, 9, 4, 1, 6, 3, 8]

Possible Output:

[8, 2, 6, 4, 1, 9, 3, 7]

ไม่จำเป็นต้องเรียงค่าภายในกลุ่มจากน้อยไปมาก

ให้ออกแบบอย่างน้อย 3 อัลกอริทึม ได้แก่

อัลกอริทึมที่ 1: Recursive Two-Pointer

ใช้ตัวชี้ left และ right

- หากตำแหน่งด้านซ้ายเป็นจำนวนคู่ ให้เลื่อน left
- หากตำแหน่งด้านขวาเป็นจำนวนคี่ ให้เลื่อน right
- หากด้านซ้ายเป็นคี่และด้านขวาเป็นคู่ ให้สลับค่า
- เรียกเมธอดแบบเวียนเกิดกับช่วงที่เหลือ

ชื่อเมธอด

static void rearrangeRecursive(int[] a, int left, int right)

1. คำอธิบายแนวคิด

ใช้ตัวชี้ 2 ตัว คือ left และ right

- หากด้านซ้ายเป็นเลขคู่ ให้เลื่อน left
- หากด้านขวาเป็นเลขคี่ ให้เลื่อน right
- ถ้าด้านซ้ายเป็นเลขคี่และด้านขวาเป็นเลขคู่ ให้สลับค่า
- เรียกเมธอดแบบ Recursion ไปยังช่วงข้อมูลที่เหลือ

2. Pseudocode

```
rearrangeRecursive(a, left, right)

if left >= right
    return

if a[left] is even
    rearrangeRecursive(a, left+1, right)

elseif a[right] is odd
    rearrangeRecursive(a, left, right-1)

else
    swap(a[left], a[right])
    rearrangeRecursive(a, left+1, right-1)
```

3. โปรแกรมภาษา Java

```
import java.util.Arrays;

public class RecursiveEvenOdd{

    static void rearrangeRecursive(int[] a, int left, int right){

        if(left >= right){
            return;
        }

        if(a[left]%2==0){
            rearrangeRecursive(a, left +1, right);
        }
        else if(a[right]%2!=0){
            rearrangeRecursive(a, left, right -1);
        }
        else{

            int temp = a[left];
            a[left] = a[right];
            a[right] = temp;

            rearrangeRecursive(a, left +1, right -1);
        }
    }

    public static void main(String[] args){

        int[] a = {7, 2, 9, 4, 1, 6, 3, 8};

        rearrangeRecursive(a, 0, a.length -1);

        System.out.println(Arrays.toString(a));
    }
}
```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input : 7, 2, 9, 4, 1, 6, 3, 8

Output : 8, 2, 6, 4, 1, 9, 3, 7

5. Time Complexity

แต่ละสมาชิกถูกตรวจสอบไม่เกิน 1 ครั้ง

Time Complexity = $O(n)$

6. Space Complexity

เกิด Recursive Call ตามจำนวนสมาชิก

Space Complexity = $O(n)$

7. ข้อดีและข้อจำกัด

ข้อดี

- ไม่ต้องใช้อาร์เรย์เพิ่ม
- ใช้แนวคิด Two-Pointer ได้ดี

ข้อจำกัด

- มีค่าใช้จ่ายจาก Recursion
- อาจเกิด StackOverflowError ถ้าข้อมูลมีขนาดใหญ่มาก

8. สรุป

เหมาะสำหรับศึกษาแนวคิด Recursion ร่วมกับ Two-Pointer

อัลกอริทึมที่ 2: Iterative Two-Pointer

ใช้หลักการเดียวกับวิธีแรก แต่ใช้ลูปแทนการเวียนเกิด

ชื่อเมธอด

static void rearrangeTwoPointer(int[] a)

1. คำอธิบายแนวคิด

ใช้หลักการเดียวกับวิธีแรก แต่ใช้ลูปแทน Recursion

- left วิ่งจากซ้าย
- right วิ่งจากขวา
- เมื่อพบเลขคี่ด้านซ้ายและเลขคู่ด้านขวาให้สลับ

2. Pseudocode

```
left = 0
right = n-1

while left < right
    while even at left
        left++

    while odd at right
        right--

    swap(left, right)
```

3. โปรแกรมภาษา Java

```
import java.util.Arrays;

public class IterativeEvenOdd{

    static void rearrangeTwoPointer(int[] a){

        int left =0;
        int right = a.length -1;

        while(left < right){

            while(left < right && a[left]%2==0){
                left++;
            }

            while(left < right && a[right]%2!=0){
                right--;
            }

            if(left < right){

                int temp = a[left];
                a[left]= a[right];
                a[right]= temp;

                left++;
                right--;
            }
        }
    }

    public static void main(String[] args){

        int[] a ={7, 2, 9, 4, 1, 6, 3, 8};

        rearrangeTwoPointer(a);

        System.out.println(Arrays.toString(a));
    }
}
```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input : 7, 2, 9, 4, 1, 6, 3, 8

Output : 8, 2, 6, 4, 1, 9, 3, 7

5. Time Complexity

$O(n)$

6. Space Complexity

$O(1)$

7. ข้อดีและข้อจำกัด

ข้อดี

- เร็ว
- ใช้หน่วยความจำต่ำ
- ไม่เกิด StackOverflowError

ข้อจำกัด

- ลำดับเดิมอาจเปลี่ยนไป

8. สรุป

เหมาะสำหรับการใช้งานจริงและข้อมูลขนาดใหญ่

อัลกอริทึมที่ 3: Extra Array

สร้างอาร์เรย์ใหม่ โดยนำจำนวนคู่ไว้ก่อน แล้วจึงนำจำนวนคี่ใส่ตามหลัง
ชื่อเมธอด

```
static int[] rearrangeExtraArray(int[] a)
```

1. คำอธิบายแนวคิด

สร้างอาร์เรย์ใหม่

- รอบแรกเก็บจำนวนคู่ทั้งหมด
- รอบที่สองเก็บจำนวนคี่ทั้งหมด

ทำให้ลำดับเดิมภายในแต่ละกลุ่มยังคงอยู่

2. Pseudocode

```
create new array result

for each number
  if even
    add to result

for each number
  if odd
    add to result

return result
```

3. โปรแกรมภาษา Java

```
import java.util.Arrays;

public class ExtraArrayEvenOdd {

    static int[] rearrangeExtraArray(int[] a){

        int[] result = new int[a.length];
        int index = 0;

        for(int num : a){
            if(num % 2 == 0){
                result[index++] = num;
            }
        }
    }
}
```

```

for(int num : a){
    if(num %2 !=0){
        result[index++] = num;
    }
}

return result;
}

public static void main(String[] args){

    int[] a = {7, 2, 9, 4, 1, 6, 3, 8};

    int[] result = rearrangeExtraArray(a);

    System.out.println(Arrays.toString(result));
}
}

```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input : 5, 2, 7, 4, 9, 6

Stable Output : 2, 4, 6, 5, 7, 9

ลำดับเดิมของเลขคู่และเลขคี่ยังคงอยู่

5. Time Complexity

ตรวจสอบข้อมูล 2 รอบ

$O(n)$

6. Space Complexity

มีการสร้างอาร์เรย์ใหม่

$O(n)$

7. ข้อดีและข้อจำกัด

ข้อดี

- เป็น Stable Algorithm
- เข้าใจง่าย
- ลำดับเดิมไม่เปลี่ยน

ข้อจำกัด

- ใช้หน่วยความจำเพิ่ม

8. สรุป

เหมาะกับงานที่ต้องรักษาลำดับเดิมของข้อมูล

งานวิเคราะห์

ให้นักศึกษาวิเคราะห์และเปรียบเทียบ

ให้ตรวจสอบว่าวิธีใดรักษาลำดับเดิมของสมาชิกได้

ตัวอย่าง

Input:

[5, 2, 7, 4, 9, 6]

Stable Output:

[2, 4, 6, 5, 7, 9]

- **Time Complexity**

อัลกอริทึม	Time Complexity
Recursive Two-Pointer	$O(n)$
Iterative Two-Pointer	$O(n)$
Extra Array	$O(n)$

- **Space Complexity**

อัลกอริทึม	Space Complexity
Recursive Two-Pointer	$O(n)$
Iterative Two-Pointer	$O(1)$
Extra Array	$O(n)$

- จำนวนครั้งของการสลับข้อมูล

ตัวอย่าง

7, 2, 9, 4, 1, 6, 3, 8

Recursive Two-Pointer

สลับประมาณ 2 ครั้ง

Iterative Two-Pointer

สลับประมาณ 2 ครั้ง

Extra Array

0 ครั้ง

เพราะคัดลอกข้อมูลลงอาร์เรย์ใหม่

- การเปลี่ยนแปลงอาร์เรย์เดิม

อัลกอริทึม	เปลี่ยนอาร์เรย์เดิม
Recursive Two-Pointer	ใช่
Iterative Two-Pointer	ใช่
Extra Array	ไม่

- ความเป็น **Stable Algorithm**

Stable หมายถึงสมาชิกที่อยู่กลุ่มเดียวกันยังคงลำดับเดิม

ตัวอย่าง

5, 2, 7, 4, 9, 6

ผลลัพธ์ที่ Stable

2, 4, 6, 5, 7, 9

อัลกอริทึม	Stable
Recursive Two-Pointer	ไม่เป็น
Iterative Two-Pointer	ไม่เป็น
Extra Array	เป็น

สรุปเปรียบเทียบ

หัวข้อ	Recursive Two-Pointer	Iterative Two-Pointer	Extra Array
Time Complexity	$O(n)$	$O(n)$	$O(n)$
Space Complexity	$O(n)$	$O(1)$	$O(n)$
จำนวนการสลับ	น้อย	น้อย	ไม่มี
เปลี่ยนอาร์เรย์เดิม	ใช่	ใช่	ไม่
Stable	ไม่	ไม่	เป็น
เหมาะกับข้อมูลใหญ่	ปานกลาง	ดีมาก	ดี

สรุปสุดท้าย

หากต้องการประสิทธิภาพสูงและใช้หน่วยความจำน้อยที่สุด ควรเลือก Iterative Two-Pointer เพราะมี Time Complexity = $O(n)$ และ Space Complexity = $O(1)$ ส่วน Extra Array เหมาะสำหรับกรณีที่ต้องรักษาลำดับเดิมของข้อมูล (Stable Algorithm) ขณะที่ Recursive Two-Pointer เหมาะสำหรับการศึกษาแนวคิด Recursion และ Divide and Conquer มากกว่า

ข้อ 5 การแบ่งอาร์เรย์ตามค่า k

กำหนดอาร์เรย์จำนวนเต็มที่ยังไม่เรียงลำดับ A และจำนวนเต็ม k
ให้นักศึกษาเขียนโปรแกรมจัดตำแหน่งสมาชิกใหม่ โดยให้

- สมาชิกที่มีค่าน้อยกว่าหรือเท่ากับ k อยู่ด้านหน้า
- สมาชิกที่มีค่ามากกว่า k อยู่ด้านหลัง

ตัวอย่าง

A = [12, 4, 7, 15, 3, 10, 8]

k = 8

ผลลัพธ์ที่เป็นไปได้

[8, 4, 7, 3, 15, 10, 12]

ให้ออกแบบอย่างน้อย 3 อัลกอริทึม ได้แก่

อัลกอริทึมที่ 1: Recursive Partition

ใช้ตัวชี้ซ้ายและขวาเพื่อตรวจสอบและสลับสมาชิกในอาร์เรย์แบบเวียนเกิด
ชื่อเมธอด

static void partitionRecursive(int[] a, int k, int left, int right)

1. คำอธิบายแนวคิด

ใช้อัลกอริทึมแบบ Recursive Two-Pointer

- ใช้ตัวชี้ซ้าย (left) และขวา (right)
- ถ้าด้านซ้ายมีค่าน้อยกว่าหรือเท่ากับ k ให้เลื่อน left
- ถ้าด้านขวามีค่ามากกว่า k ให้เลื่อน right
- ถ้าพบค่าผิดฝั่งทั้งสองด้านให้สลับตำแหน่ง
- เรียกเมธอดแบบเวียนเกิดกับช่วงข้อมูลที่เหลือ

2. Pseudocode

```
partitionRecursive(a, k, left, right)

if left >= right
    return

if a[left] <= k
    partitionRecursive(a, k, left+1, right)

elseif a[right] > k
    partitionRecursive(a, k, left, right-1)

else
    swap(a[left], a[right])
    partitionRecursive(a, k, left+1, right-1)
```

3. โปรแกรมภาษา Java

```
import java.util.Arrays;

public class RecursivePartition{

    static void partitionRecursive(int[] a,
    int k,
    int left,
    int right){

        if(left >= right){
            return;
        }

        if(a[left]<= k){
            partitionRecursive(a, k, left +1, right);
        }
        elseif(a[right]> k){
            partitionRecursive(a, k, left, right -1);
        }
        else{

            int temp = a[left];
            a[left]= a[right];
            a[right]= temp;

            partitionRecursive(a, k,
            left +1,
            right -1);
        }
    }

    public static void main(String[] args){

        int[] a ={12, 4, 7, 15, 3, 10, 8};
        int k =8;

        partitionRecursive(a, k, 0, a.length -1);

        System.out.println(Arrays.toString(a));
    }
}
```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input : A = 12, 4, 7, 15, 3, 10, 8

k = 8

Output : 8, 4, 7, 3, 15, 10, 12

5. Time Complexity

$O(n)$

6. Space Complexity

$O(n)$

7. ข้อดีและข้อจำกัด

ข้อดี

- ไม่ต้องใช้อาร์เรย์เพิ่ม
- ใช้หลักการเดียวกับ Partition ของ Quick Sort

ข้อจำกัด

- ใช้ Call Stack
- อาจเกิด StackOverflowError เมื่อข้อมูลใหญ่มาก

8. สรุป

เหมาะสำหรับศึกษาแนวคิด Recursion และ Partition Process

อัลกอริทึมที่ 2: Iterative Partition

ใช้ลูปและตัวชี้สองตำแหน่งเพื่อแบ่งอาร์เรย์
ชื่อเมธอด

static void partitionIterative(int[] a, int k)

1. คำอธิบายแนวคิด

ใช้ตัวชี้สองตำแหน่ง

- left ค้นหาค่าที่มากกว่า k
- right ค้นหาค่าน้อยกว่าหรือเท่ากับ k
- เมื่อพบให้สลับข้อมูล
- ทำซ้ำจนตัวชี้ตัดกัน

2. Pseudocode

```
left = 0
right = n - 1

while left < right

while a[left] <= k
left++

while a[right] > k
right--

swap(a[left], a[right])
```

3. โปรแกรมภาษา Java

```
import java.util.Arrays;

public class IterativePartition{

    static void partitionIterative(int[] a, int k){

        int left =0;
        int right = a.length -1;

        while(left < right){

            while(left < right &&
a[left]<= k){
                left++;
            }

            while(left < right &&
a[right]> k){
                right--;
            }

            if(left < right){

                int temp = a[left];
                a[left]= a[right];
                a[right]= temp;

                left++;
                right--;
            }
        }

        public static void main(String[] args){

            int[] a ={12, 4, 7, 15, 3, 10, 8};
            int k =8;

            partitionIterative(a, k);

            System.out.println(Arrays.toString(a));
        }
    }
}
```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input : A = [12, 4, 7, 15, 3, 10, 8]

k = 8

Output : 8, 4, 7, 3, 15, 10, 12

5. Time Complexity

$O(n)$

6. Space Complexity

$O(1)$

7. ข้อดีและข้อจำกัด

ข้อดี

- ทำงานเร็ว
- ใช้หน่วยความจำน้อย
- ไม่เกิด StackOverflowError

ข้อจำกัด

- ไม่รักษาลำดับเดิมของสมาชิก

8. สรุป

เหมาะสำหรับการใช้งานจริงและข้อมูลขนาดใหญ่

อัลกอริทึมที่ 3: Sorting-Based Algorithm

เรียงอาร์เรย์ก่อน แล้วค้นหาตำแหน่งสุดท้ายที่มีค่าน้อยกว่าหรือเท่ากับ k

ชื่อเมธอด

```
static void partitionBySorting(int[] a, int k)
```

1. คำอธิบายแนวคิด

เรียงอาร์เรย์จากน้อยไปมากก่อน จากนั้นข้อมูลที่มีค่าน้อยกว่าหรือเท่ากับ k จะอยู่ด้านหน้าโดยอัตโนมัติ

วิธีนี้สามารถแก้ปัญหาได้ แต่มีการทำงานมากเกินไปจนความจำเป็น

2. Pseudocode

```
sort array  
  
find boundary position  
  
elements <= k  
stay in front  
  
elements > k  
stay behind
```

3. โปรแกรมภาษา Java

```
import java.util.Arrays;

public class SortingPartition{

    static void partitionBySorting(int[] a, int k){

        Arrays.sort(a);
    }

    public static void main(String[] args){

        int[] a = {12, 4, 7, 15, 3, 10, 8};
        int k = 8;

        partitionBySorting(a, k);

        System.out.println(Arrays.toString(a));
    }
}
```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input : A = 12, 4, 7, 15, 3, 10, 8

k = 8

Output : 3, 4, 7, 8, 10, 12, 15

5. Time Complexity

การเรียงข้อมูลด้วย Arrays.sort()

$O(n \log n)$

6. Space Complexity

$O(\log n)$

(จากการทำงานภายในของ Arrays.sort)

7. ข้อดีและข้อจำกัด

ข้อดี

- ได้ข้อมูลเรียงลำดับด้วย
- เหมาะเมื่อจำเป็นต้องใช้ข้อมูลที่เรียงแล้วต่อ

ข้อจำกัด

- ช้ากว่าวิธี Partition
- ทำงานเกินความจำเป็นสำหรับโจทย์นี้

8. สรุป

ไม่เหมาะหากต้องการเพียงแบ่งข้อมูลตามค่า k

งานวิเคราะห์

ให้นักศึกษาวิเคราะห์

- **Time Complexity** ของแต่ละวิธี
- **Space Complexity** ของแต่ละวิธี
- **วิธี Recursive Partition**
 - ใช้ Two-Pointer แบบ Recursion
 - ทำงาน In-place
 - ไม่ใช้อาร์เรย์เพิ่ม
 - ใช้ Call Stack เพิ่ม

Time Complexity

$O(n)$

Space Complexity

$O(n)$

- **วิธี Iterative Partition**
 - ใช้ Two-Pointer และ Loop
 - ทำงาน In-place
 - ไม่ใช้อาร์เรย์เพิ่ม

Time Complexity

$O(n)$

Space Complexity

$O(1)$

- **วิธี Sorting-Based**
 - เรียงข้อมูลทั้งอาร์เรย์
 - ได้ผลลัพธ์ที่ถูกต้องแต่ไม่จำเป็น

Time Complexity

$O(n \log n)$

Space Complexity

$O(\log n)$

- เหตุผลที่การเรียงลำดับอาจทำให้โปรแกรมช้ากว่าที่จำเป็น

โจทย์ต้องการเพียง

$\leq k \mid > k$

แต่การเรียงลำดับต้องเปรียบเทียบสมาชิกจำนวนมากเพื่อจัดลำดับทุกตัว

ตัวอย่าง

12,4,7,15,3,10,8

เราสามารถแบ่งกลุ่มได้โดยไม่จำเป็นต้องรู้ว่า

$3 < 4 < 7 < 8$

ดังนั้นการ Sort จึงเป็นงานที่เกินความจำเป็น

- ความสัมพันธ์ของปัญหานี้กับขั้นตอน Partition ใน Quick Sort

วิธี Recursive Partition และ Iterative Partition ใช้แนวคิดเดียวกับขั้นตอน Partition ของ

Quick Sort

หลักการคือ

ค่ากลุ่มหนึ่งอยู่ซ้าย

ค่ากลุ่มหนึ่งอยู่ขวา

โดยไม่จำเป็นต้องเรียงลำดับภายในกลุ่ม จึงช่วยลดเวลาการทำงานจาก

$O(n \log n)$

เหลือ

$O(n)$

สำหรับการแบ่งกลุ่มเพียงครั้งเดียว

ให้นักศึกษาระบุด้วยว่าอัลกอริทึมใดสามารถทำงานแบบ In-place ได้

อัลกอริทึม	In-place
Recursive Partition	ใช่
Iterative Partition	ใช่
Sorting-Based	ใช่

ทั้ง 3 วิธีสามารถทำงานบนอาร์เรย์เดิมได้โดยไม่ต้องสร้างอาร์เรย์ใหม่ขนาด n

สรุปเปรียบเทียบ

หัวข้อ	Recursive Partition	Iterative Partition	Sorting-Based
Time Complexity	$O(n)$	$O(n)$	$O(n \log n)$
Space Complexity	$O(n)$	$O(1)$	$O(\log n)$
In-place	ใช่	ใช่	ใช่
ใช้ Recursion	ใช่	ไม่ใช่	ไม่ใช่
เหมาะกับข้อมูลขนาดใหญ่	ปานกลาง	ดีมาก	ดี
ประสิทธิภาพสำหรับโจทย์นี้	ดี	ดีที่สุด	ต่ำสุด

สรุปสุดท้าย

สำหรับโจทย์การแบ่งอาร์เรย์ตามค่า k วิธีที่เหมาะสมที่สุดคือ Iterative Partition เพราะมี Time Complexity = $O(n)$, Space Complexity = $O(1)$ และทำงานแบบ In-place ส่วน Recursive Partition เหมาะสำหรับการศึกษาแนวคิด Partition และ Quick Sort ขณะที่ Sorting-Based Algorithm แม้จะให้ผลลัพธ์ถูกต้องแต่มีการทำงานเกินความจำเป็นและใช้เวลามากกว่า จึงไม่เหมาะกับโจทย์นี้มากนัก

ข้อ 6 การค้นหาจำนวนที่มีผลรวมเท่ากับ k

กำหนดอาร์เรย์ A ที่มีจำนวนเต็มไม่ซ้ำกันจำนวน n ค่า และสมาชิกเรียงจากน้อยไปมากแล้ว พร้อมจำนวนเต็ม k

ให้นักศึกษาเขียนโปรแกรมค้นหาสมาชิกสองค่าที่มีผลรวมเท่ากับ k

ตัวอย่าง

$A = [2, 4, 7, 11, 15, 20]$

$k = 18$

ผลลัพธ์

Pair found: 7 and 11

ให้ออกแบบอย่างน้อย 3 อัลกอริทึม ได้แก่

อัลกอริทึมที่ 1: Brute Force

ตรวจสอบสมาชิกทุกคู่ที่เป็นไปได้

ชื่อเมธอด

```
static boolean findPairBruteForce(int[] a, int k)
```

1. คำอธิบายแนวคิด

ตรวจสอบสมาชิกทุกคู่ที่เป็นไปได้

- เลือกสมาชิกตัวแรก
- เปรียบเทียบกับสมาชิกทุกตัวที่เหลือ
- หากผลรวมเท่ากับ k ให้รายงานผล

2. Pseudocode

```
for i = 0 to n-1
  for j = i+1 to n-1
    if a[i] + a[j] == k
      return true
  return false
```

3. โปรแกรมภาษา Java

```
public class PairBruteForce {  
  
    static boolean findPairBruteForce(int[] a, int k){  
  
        for(int i =0; i < a.length; i++){  
  
            for(int j = i +1; j < a.length; j++){  
  
                if(a[i]+ a[j]== k){  
  
                    System.out.println(  
                        "Pair found:"  
                        + a[i]  
                        +"and "  
                        + a[j]);  
  
                    return true;  
                }  
            }  
        }  
  
        return false;  
    }  
  
    public static void main(String[] args){  
  
        int[] a ={2, 4, 7, 11, 15, 20};  
  
        findPairBruteForce(a, 18);  
    }  
}
```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input : A = 2, 4, 7, 11, 15, 20

k = 18

Output : Pair found: 7 and 11

5. Time Complexity

จำนวนคู่ทั้งหมด

$$n(n-1)/2$$

ดังนั้น

$$\text{Time Complexity} = O(n^2)$$

6. Space Complexity

$$\text{Space Complexity} = O(1)$$

7. ข้อดีและข้อจำกัด

ข้อดี

- เข้าใจง่าย
- ใช้ได้กับข้อมูลที่ไม่เรียงลำดับ

ข้อจำกัด

- ช้ามากเมื่อข้อมูลมีขนาดใหญ่

8. สรุป

เหมาะกับข้อมูลขนาดเล็กและการศึกษาหลักการพื้นฐาน

อัลกอริทึมที่ 2: Recursive Two-Pointer

กำหนดตัวชี้สองตำแหน่ง

left = 0

right = n - 1

คำนวณผลรวม

$A[\text{left}] + A[\text{right}]$

แล้วดำเนินการดังนี้

- หากผลรวมเท่ากับ k ให้รายงานคู่ที่พบ
- หากผลรวมน้อยกว่า k ให้เพิ่มค่า left
- หากผลรวมมากกว่า k ให้ลดค่า right
- เรียกเมธอดแบบเวียนเกิดกับช่วงใหม่

ชื่อเมธอด

```
static boolean findPairRecursive( int[] a, int k, int left, int right)
```

1. คำอธิบายแนวคิด

เนื่องจากอาร์เรย์เรียงลำดับจากน้อยไปมากแล้ว

กำหนด

left = 0

right = n - 1

คำนวณ

$\text{sum} = A[\text{left}] + A[\text{right}]$

- ถ้า $\text{sum} = k$ พบคำตอบ
- ถ้า $\text{sum} < k$ เลื่อน left
- ถ้า $\text{sum} > k$ เลื่อน right
- เรียกเมธอดแบบ Recursion ต่อ

2. Pseudocode

```
findPairRecursive(a,k,left,right)

if left >= right
return false

sum = a[left]+ a[right]

if sum == k
return true

if sum < k
return recursion(left+1,right)

return recursion(left,right-1)
```

3. โปรแกรมภาษา Java

```
public class PairRecursive {

static boolean findPairRecursive(
int[] a,
int k,
int left,
int right){

if(left >= right){
return false;
}

int sum = a[left]+ a[right];

if(sum == k){

System.out.println(
"Pair found:"
+ a[left]
+"and "
+ a[right]);

return true;
}

if(sum < k){
return findPairRecursive(
a,
k,
left +1,
right);
}

return findPairRecursive(
a,
k,
left,
right -1);
}

public static void main(String[] args){

int[] a = {2, 4, 7, 11, 15, 20};
```

```
findPairRecursive(  
a,  
18,  
0,  
a.length - 1);  
}  
}
```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input : A = [2, 4, 7, 11, 15, 20]

k = 18

Output : Pair found: 7 and 11

5. Time Complexity

แต่ละรอบเลื่อน pointer อย่างน้อย 1 ตำแหน่ง

Time Complexity = $O(n)$

6. Space Complexity

เกิด Recursive Call หลายชั้น

Space Complexity = $O(n)$

7. ข้อดีและข้อจำกัด

ข้อดี

- เร็วกว่า Brute Force
- ใช้คุณสมบัติของข้อมูลที่เรียงแล้ว
- ลดจำนวนการตรวจสอบลงมาก

ข้อจำกัด

- ต้องใช้อาร์เรย์ที่เรียงแล้ว
- มีค่าใช้จ่ายจาก Recursion

8. สรุป

เหมาะกับข้อมูลที่เรียงลำดับแล้วและต้องการลดเวลาในการค้นหา

อัลกอริทึมที่ 3: Binary Search

เลือกสมาชิก $A[i]$ ที่ละตัว แล้วใช้ Binary Search ค้นหาค่า

$$k - A[i]$$

ในสมาชิกที่เหลือ

ชื่อเมธอด

static boolean findPairBinarySearch(int[] a, int k)

1. คำอธิบายแนวคิด

เลือกสมาชิก $A[i]$ ที่ละตัว

จากนั้นหาค่า

$$\text{target} = k - A[i]$$

แล้วใช้ Binary Search ค้นหา target ในส่วนที่เหลือของอาร์เรย์

2. Pseudocode

```
for each element
    target = k - a[i]
    binary search target
    if found
        return true
```

3. โปรแกรมภาษา Java

```
public class PairBinarySearch {

    static boolean findPairBinarySearch(
        int[] a,
        int k) {

        for (int i = 0; i < a.length; i++) {

            int target = k - a[i];

            int left = i + 1;
            int right = a.length - 1;

            while (left <= right) {

                int mid =
                    (left + right) / 2;

                if (a[mid] == target) {

                    System.out.println(
                        "Pair found:"
                        + a[i]
                        + " and "
                        + a[mid]);

                    return true;
                }
            }
        }
    }
}
```

```

if(a[mid]< target){
    left = mid +1;
}else{
    right = mid -1;
}
}
}

return false;
}

public static void main(String[] args){
    int[] a ={2, 4, 7, 11, 15, 20};
    findPairBinarySearch(a, 18);
}
}

```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input : A = 2, 4, 7, 11, 15, 20

k = 18

Output : Pair found: 7 and 11

5. Time Complexity

สำหรับสมาชิกแต่ละตัว

Binary Search = $O(\log n)$

ทำทั้งหมด n รอบ

Time Complexity = $O(n \log n)$

6. Space Complexity

Space Complexity = $O(1)$

..

7. ข้อดีและข้อจำกัด

ข้อดี

- ใช้ประโยชน์จากข้อมูลที่เรียงแล้ว
- เร็วกว่า Brute Force

ข้อจำกัด

- ช้ากว่า Two-Pointer
- เขียนซับซ้อนกว่า

8. สรุป

เหมาะกับงานค้นหาค้นหาอาร์เรย์ที่เรียงแล้วและต้องการใช้ Binary Search

งานวิเคราะห์

ให้นักศึกษาวิเคราะห์และเปรียบเทียบ

อัลกอริทึม	แนวคิด	Time Complexity	Space Complexity
Brute Force	ตรวจสอบทุกคู่	ให้นักศึกษาวิเคราะห์	ให้นักศึกษาวิเคราะห์
Recursive Two-Pointer	ลดขอบเขตการค้นหาที่ละตำแหน่ง	ให้นักศึกษาวิเคราะห์	ให้นักศึกษาวิเคราะห์
Binary Search	ค้นหาค่าคู่สมของสมาชิกแต่ละตัว	ให้นักศึกษาวิเคราะห์	ให้นักศึกษาวิเคราะห์

ให้อธิบายว่าเหตุใด Two-Pointer จึงใช้ได้เมื่ออาร์เรย์เรียงลำดับแล้ว และจะเกิดอะไรขึ้นหากนำวิธีนี้ไปใช้กับอาร์เรย์ที่ยังไม่เรียงลำดับ

งานวิเคราะห์และเปรียบเทียบ

อัลกอริทึม	แนวคิด	Time Complexity	Space Complexity
Brute Force	ตรวจสอบทุกคู่	$O(n^2)$	$O(1)$
Recursive Two-Pointer	ลดขอบเขตการค้นหาที่ละตำแหน่ง	$O(n)$	$O(n)$
Binary Search	ค้นหาคู่ของสมาชิกแต่ละตัว	$O(n \log n)$	$O(1)$

เหตุใด Two-Pointer จึงใช้ได้เมื่ออาร์เรย์เรียงลำดับแล้ว

ตัวอย่าง

$A = [2, 4, 7, 11, 15, 20]$

$k = 18$

เริ่มต้น

$2 + 20 = 22$

ผลรวมมากกว่า 18 จึงลดค่า right ลงได้ทันที เพราะสมาชิกทางขวาทั้งหมดมีค่ามากกว่า 20 ไม่จำเป็นต้องตรวจสอบอีก

ต่อมา

$2 + 15 = 17$

ผลรวมน้อยกว่า 18

จึงเพิ่ม left ได้ทันที

การเรียงลำดับทำให้สามารถตัดตัวเลือกที่ไม่จำเป็นออกได้ในแต่ละรอบ

ถ้าใช้อาร์เรย์ที่ยังไม่เรียงลำดับจะเกิดอะไรขึ้น

ตัวอย่าง

11, 2, 20, 4, 15, 7

หากเลื่อน left หรือ right

1. ไม่สามารถทราบได้ว่า
2. ผลรวมจะเพิ่มขึ้นหรือลดลง

เพราะข้อมูลไม่ได้เรียงลำดับ

ดังนั้น Two-Pointer จึงไม่สามารถรับประกันความถูกต้องได้กับอาร์เรย์ที่ไม่เรียงลำดับ

สรุปเปรียบเทียบ

หัวข้อ	Brute Force	Recursive Two-Pointer	Binary Search
Time Complexity	$O(n^2)$	$O(n)$	$O(n \log n)$
Space Complexity	$O(1)$	$O(n)$	$O(1)$
ใช้ข้อมูลเรียงลำดับ	ไม่จำเป็น	จำเป็น	จำเป็น
ความเร็ว	ช้าที่สุด	เร็วที่สุด	ปานกลาง
ความง่าย	ง่าย	ปานกลาง	ปานกลาง

สรุปสุดท้าย

สำหรับโจทย์นี้ Recursive Two-Pointer เป็นอัลกอริทึมที่มีประสิทธิภาพดีที่สุด เพราะใช้เวลาเพียง $O(n)$ โดยอาศัยคุณสมบัติที่ข้อมูลถูกเรียงลำดับไว้แล้ว ส่วน Binary Search มีประสิทธิภาพรองลงมา ($O(n \log n)$) และ Brute Force ช้าที่สุด ($O(n^2)$) เนื่องจากต้องตรวจสอบทุกคู่ที่เป็นไปได้ทั้งหมด

งานทดลองเปรียบเทียบประสิทธิภาพ

ให้นักศึกษาเลือกอย่างน้อย 2 ข้อจากแบบฝึกหัดข้างต้น แล้วทดลองวัดเวลาการทำงานจริง กำหนดขนาดข้อมูลอย่างน้อย 4 ระดับ เช่น

n = 100

n = 1,000

n = 10,000

n = 100,000

ใช้คำสั่งต่อไปนี้ในการวัดเวลา

```
long start = System.nanoTime();

// เรียกใช้อัลกอริทึม

long end = System.nanoTime();

System.out.println(
    "Execution time:"+(end - start)+"nanoseconds"
);
```

เพื่อให้ผลมีความน่าเชื่อถือ ควรทดลองแต่ละขนาดอย่างน้อย 5 ครั้ง แล้วคำนวณเวลาเฉลี่ย

Average Time = ผลรวมเวลาจากการทดลองทั้งหมด ÷ จำนวนครั้งที่ทดลอง

ให้นำเสนอผลในตาราง

ขนาดข้อมูล n	Algorithm 1	Algorithm 2	Algorithm 3
100			
1,000			
10,000			
100,000			

โปรแกรมทดลองเวลา (Template)

```
long totalTime = 0;

for(int i = 0; i < 5; i++){

    long start = System.nanoTime();

    // เรียกใช้อัลกอริทึม

    long end = System.nanoTime();

    totalTime += (end - start);
}

double averageTime = totalTime / 5.0;

System.out.println(
    "Average Time ="
    + averageTime
    + "nanoseconds");
```

ทดลองจริงด้วยการรันโค้ดเปรียบเทียบ ข้อ 6 การค้นหาจำนวนที่มีผลรวมเท่ากับ k โดยใช้ข้อมูลเรียงลำดับ และกำหนด k = -1 เพื่อให้ไม่พบคำตอบ (ใกล้เคียง Worst Case) วัดเวลา 5 ครั้งแล้วหาค่าเฉลี่ย

ผลการทดลอง

ขนาดข้อมูล (n)	Brute Force	Recursive/Two-Pointer*	Binary Search
100	494,007 ns	5,067 ns	150,688 ns
1,000	69,174,501 ns	63,654 ns	1,627,112 ns
10,000	4,190,457,659 ns	666,982 ns	11,086,093 ns

ในการทดลองใช้ Two-Pointer แบบวนลูปซึ่งมีประสิทธิภาพใกล้เคียงกับ Recursive Two-Pointer แต่ไม่มีค่าใช้จ่ายจาก Call Stack

จากนั้นตอบคำถามต่อไปนี้

1. ผลการทดลองสอดคล้องกับ Big-O ที่วิเคราะห์ไว้หรือไม่

สอดคล้อง

- Brute Force $\rightarrow O(n^2)$
- Two-Pointer $\rightarrow O(n)$
- Binary Search $\rightarrow O(n \log n)$

เมื่อ n เพิ่มขึ้น

Brute Force

494,007

*

69,174,501

↓

4,190,457,659

เวลาเพิ่มขึ้นรุนแรงมากตามลักษณะ $O(n^2)$

2. อัลกอริทึมใดเร็วที่สุดเมื่อข้อมูลมีขนาดเล็ก

จากการทดลอง

$n = 100$

*wo-Pointer = 5,067 ns

Binary*Search = 150,688 ns

Brute Force = *94,007 ns

Two-Pointer เร็วที่สุด

3. อัลกอริทึมใดเหมาะสมที่สุดเมื่อข้อมูลมีขนาดใหญ่

เมื่อ $n = 10,000$

ผลลัพธ์

Two-Pointer = 666,98* ns

Binary Search = 11,086,093 ns*

Brute Force = 4,190,457,659 ns

Two-Pointer มีประสิทธิภาพดีที่สุด

เพราะมี

Time Complexity* = $O(n)$

4. Recursive Algorithm มีข้อจำกัดด้านหน่วยความจำอย่างไร

Recursive Algorithm

findPairRecursive*a,k,left,right)

จะสร้าง **Call Stack** ทุกครั้งที่เรียกตัวเอง

กรณีข้อมูลขนาดใหญ่

100,000+

อาจทำให้เกิด

StackOverfl*WError

ได้ในขณะที่ Iterative ไม่มีปัญหานี้

5. เหตุใดอัลกอริทึมที่มี Big-O เท่ากันจึงอาจใช้เวลาจริงแตกต่างกัน

ตัวอย่าง

Recursive Two-Pointer*

$O(n)$

Iterative Two-Pointer

$O(n)$

แม้ Big-O เท่ากัน แต่ Recursive

- มีการเรียกเมธอดซ้ำ
- มีการจัดการ Call Stack

จึงมักช้ากว่า Iterative

ดังนั้น Big-O แสดงเฉพาะแนวโน้มการเติบโต ไม่ได้แสดงเวลาจริงทั้งหมด

6. การวัดเวลาเพียงครั้งเดียวเพียงพอหรือไม่ เพราะเหตุใด

ไม่เพียงพอ

เพราะอาจได้รับผลกระทบจาก

- JVM Warm-up
- Garbage Collection
- CPU Scheduling
- โปรแกรมอื่นในระบบ

ดังนั้นควรวัดอย่างน้อย

5 ครั้ง

แล้วนำมาหาค่าเฉลี่ย

$AverageTime = \frac{\text{ผลรวมเวลาทั้งหมด}}{\text{จำนวนครั้ง}}$

สรุปผลการทดลอง

ผลการทดลองยืนยันว่า

Two-Pointer > Binary Search > *Brute*Force

ในด้านประสิทธิภาพการทำงาน

โดย Two-Pointer มี Time Complexity เท่ากับ $O(n)$ จึงเหมาะสมที่สุดสำหรับอาร์เรย์ที่เรียงลำดับแล้ว ส่วน Brute Force มี Time Complexity เท่ากับ $O(n^2)$ ทำให้เวลาเพิ่มขึ้นอย่างรวดเร็วเมื่อขนาดข้อมูลใหญ่ขึ้น และ Binary Search มีประสิทธิภาพอยู่ระหว่างสองวิธีดังกล่าวด้วย Time Complexity $O(n \log n)$ ดังนั้นสำหรับโจทย์นี้ Two-Pointer เป็นอัลกอริทึมที่เหมาะสมที่สุดในเชิงทฤษฎีและจากผลการทดลองจริง

ข้อกำหนดในการเขียนโปรแกรม

1. แต่ละอัลกอริทึมต้องเขียนเป็นเมธอดแยกกัน
2. ห้ามใช้เมธอดสำเร็จรูปที่แก้ปัญหาโดยตรง เว้นแต่โจทย์อนุญาต
3. ตั้งชื่อตัวแปรและเมธอดให้สื่อความหมาย
4. เขียน Comment อธิบายส่วนสำคัญ
5. ทดสอบกรณีปกติและกรณีพิเศษ
6. โปรแกรมต้องไม่หยุดทำงานผิดปกติเมื่อรับข้อมูลว่าง
7. ต้องอธิบาย Base Case และ Recursive Case ของเมธอดเวียนเกิด
8. ผลการวิเคราะห์ Big-O ต้องมีเหตุผลสนับสนุน ไม่ให้ตอบเฉพาะสัญลักษณ์